



SCHOOL OF COMPUTING

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

UNIT-II– S11BLH21 Programming in python

OBJECT ORIENTED CONCEPTS

Class – Objects – Constructors –Getters and Setters- Polymorphism – Encapsulation - Inheritance -Data Abstraction- Method Overloading-Method overriding-Data Hiding-Import Class

Classes and Methods

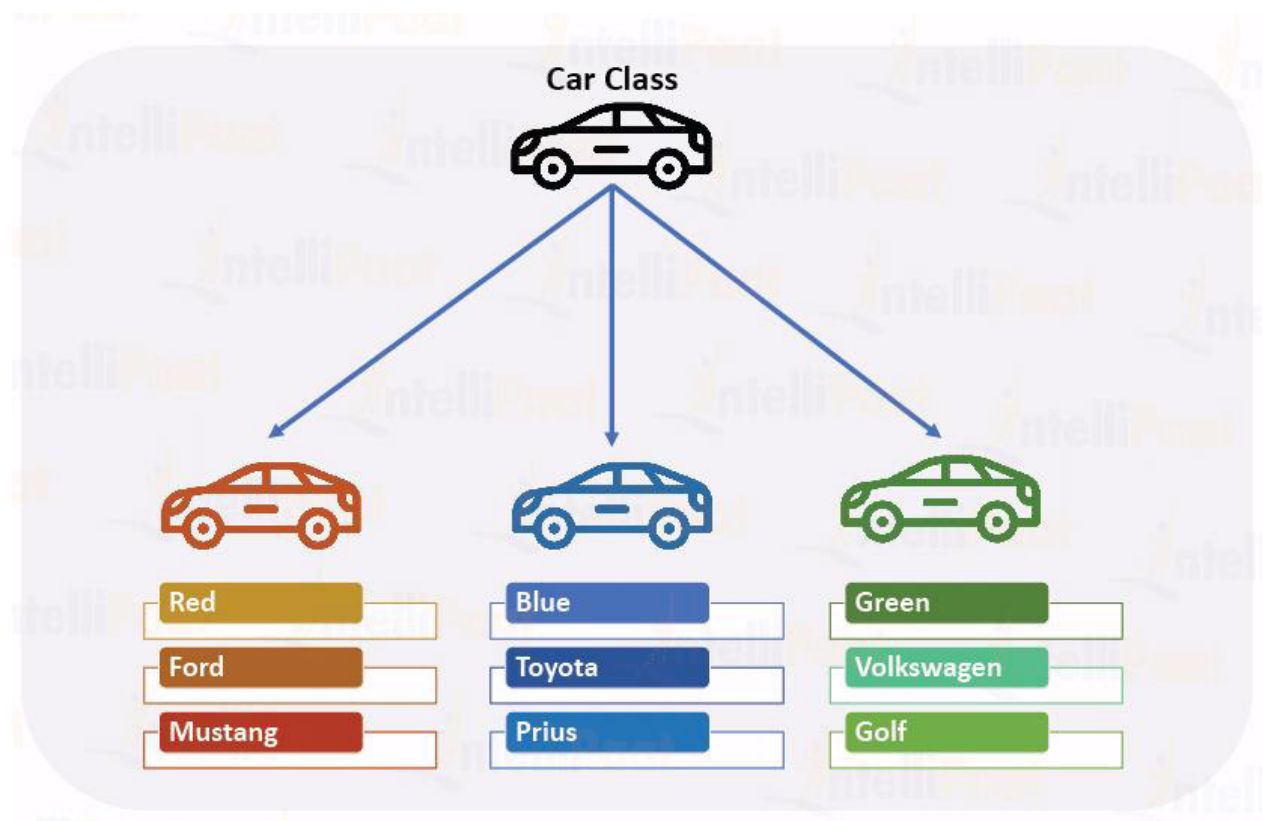
Python is an “object-oriented programming language.” This means that almost all the code is implemented using a special construct called classes. Programmers use classes to keep related things together. This is done using the keyword “class,” which is a grouping of object-oriented constructs.

What is a class?

A class is a code template for creating objects. Objects have member variables and have behaviour associated with them. In python a class is created by the keyword class.

An object is created using the constructor of the class. This object will then be called the instance of the class. In Python we create instances in the following manner

```
Instance = class(arguments)
```



How to create a class

The simplest class can be created using the class keyword. For example, let's create a simple, empty class with no functionalities.

```
>>> class Snake:
...     pass
...
>>> snake = Snake()
>>> print(snake)
<__main__.Snake object at 0x7f315c573550>
```

Attributes and Methods in class:

A class by itself is of no use unless there is some functionality associated with it. Functionalities are defined by setting attributes, which act as containers for data and functions related to those attributes. Those functions are called methods.

Attributes:

You can define the following class with the name Snake. This class will have an attribute name.

```
>>> class Snake:
...     name = "python" # set an attribute `name` of the class
...
```

You can assign the class to a variable. This is called object instantiation. You will then be able to access the attributes that are present inside the class using the dot . operator. For example, in the Snake example, you can access the attribute name of the class Snake.

```
>>> # instantiate the class Snake and assign it to variable snake
>>> snake = Snake()

>>> # access the class attribute name inside the class Snake.
>>> print(snake.name)
python
```

Methods

Once there are attributes that “belong” to the class, you can define functions that will access the class attribute. These functions are called methods. When you define methods, you will need to always provide the first argument to the method with a self keyword.

For example, you can define a class Snake, which has one attribute name and one method change_name. The method change_name will take in an argument new_name along with the keyword self.

```
>>> class Snake:
...     name = "python"
...
...     def change_name(self, new_name): # note that the first argument is self
...         self.name = new_name # access the class attribute with the self keyword
...
```

Now, you can instantiate this class Snake with a variable snake and then change the name with the method change_name.

```
>>> # instantiate the class
>>> snake = Snake()

>>> # print the current object name
>>> print(snake.name)
python

>>> # change the name using the change_name method
>>> snake.change_name("anaconda")
>>> print(snake.name)
anaconda
```

Instance attributes in python and the init method

You can also provide the values for the attributes at runtime. This is done by defining the attributes inside the init method. The following example illustrates this.

```
class Snake:

    def __init__(self, name):
        self.name = name

    def change_name(self, new_name):
```

```
self.name = new_name
```

Now you can directly define separate attribute values for separate objects. For example,

```
>>> # two variables are instantiated
>>> python = Snake("python")
>>> anaconda = Snake("anaconda")

>>> # print the names of the two variables
>>> print(python.name)
python
>>> print(anaconda.name)
anaconda
```

Python inheritance

What is Inheritance

In inheritance an object is based on another object. When inheritance is implemented, the methods and attributes that were defined in the base class will also be present in the inherited class. This is generally done to abstract away similar code in multiple classes. The abstracted code will reside in the base class and the previous classes will now inherit from the base class.

How to achieve Inheritance in Python

Python allows the classes to inherit commonly used attributes and methods from other classes through inheritance. We can define a base class in the following manner:

```
class DerivedClassName(BaseClassName):
    pass
```

Let's look at an example of inheritance. In the following example, Rocket is the base class and MarsRover is the inherited class.

```
class Rocket:
    def __init__(self, name, distance):
        self.name = name
        self.distance = distance
```

```
def launch(self):
    return "%s has reached %s" % (self.name, self.distance)

class MarsRover(Rocket): # inheriting from the base class
    def __init__(self, name, distance, maker):
        Rocket.__init__(self, name, distance)
        self.maker = maker

    def get_maker(self):
        return "%s Launched by %s" % (self.name, self.maker)

if __name__ == "__main__":
    x = Rocket("simple rocket", "till stratosphere")
    y = MarsRover("mars_rover", "till Mars", "ISRO")
    print(x.launch())
    print(y.launch())
    print(y.get_maker())
```

The output of the code above is shown below:

```
→ Documents python rockets.py
simple rocket has reached till stratosphere
mars_rover has reached till Mars
mars_rover Launched by ISRO
```

Constructor in Python

In object-oriented programming, **A constructor is a special method used to create and initialize an object of a class**. This method is defined in the class.

- The constructor is executed automatically at the time of object creation.
- The primary use of a constructor is to declare and initialize data member/ instance variables of a class. The constructor contains a collection of statements (i.e., instructions) that executes at the time of object creation to initialize the attributes of an object.

For example, when we execute `obj = Sample()`, Python gets to know that `obj` is an object of class `Sample` and calls the constructor of that class to create an object.

Note: In Python, internally, the `__new__` is the method that creates the object, and `__del__` method is called to destroy the object when the reference count for that object becomes zero.

In Python, Object creation is divided into two parts in **Object Creation** and **Object initialization**

- Internally, the `__new__` is the method that creates the object
- And, using the `__init__()` method we can implement constructor to initialize the object.

Syntax of a constructor

```
def __init__(self):  
    # body of the constructor
```

Where,

- `def`: The keyword is used to define function.
- `__init__()` Method: It is a reserved method. This method gets called as soon as an object of a class is instantiated.
- `self`: The first argument `self` refers to the current object. It binds the instance to the `__init__()` method. It's usually named `self` to follow the naming convention.

Note: The `__init__()` method arguments are optional. We can define a constructor with any number of arguments.

Example: Create a Constructor in Python

In this example, we'll create a Class **Student** with an instance variable `student name`. we'll see how to use a constructor to initialize the student name at the time of object creation.

class Student:

```
# constructor  
# initialize instance variable  
def __init__(self, name):  
    print('Inside Constructor')  
    self.name = name  
    print('All variables initialized')  
  
# instance Method  
def show(self):  
    print('Hello, my name is', self.name)
```

```
# create object using constructor  
s1 = Student('Emma')  
s1.show()
```

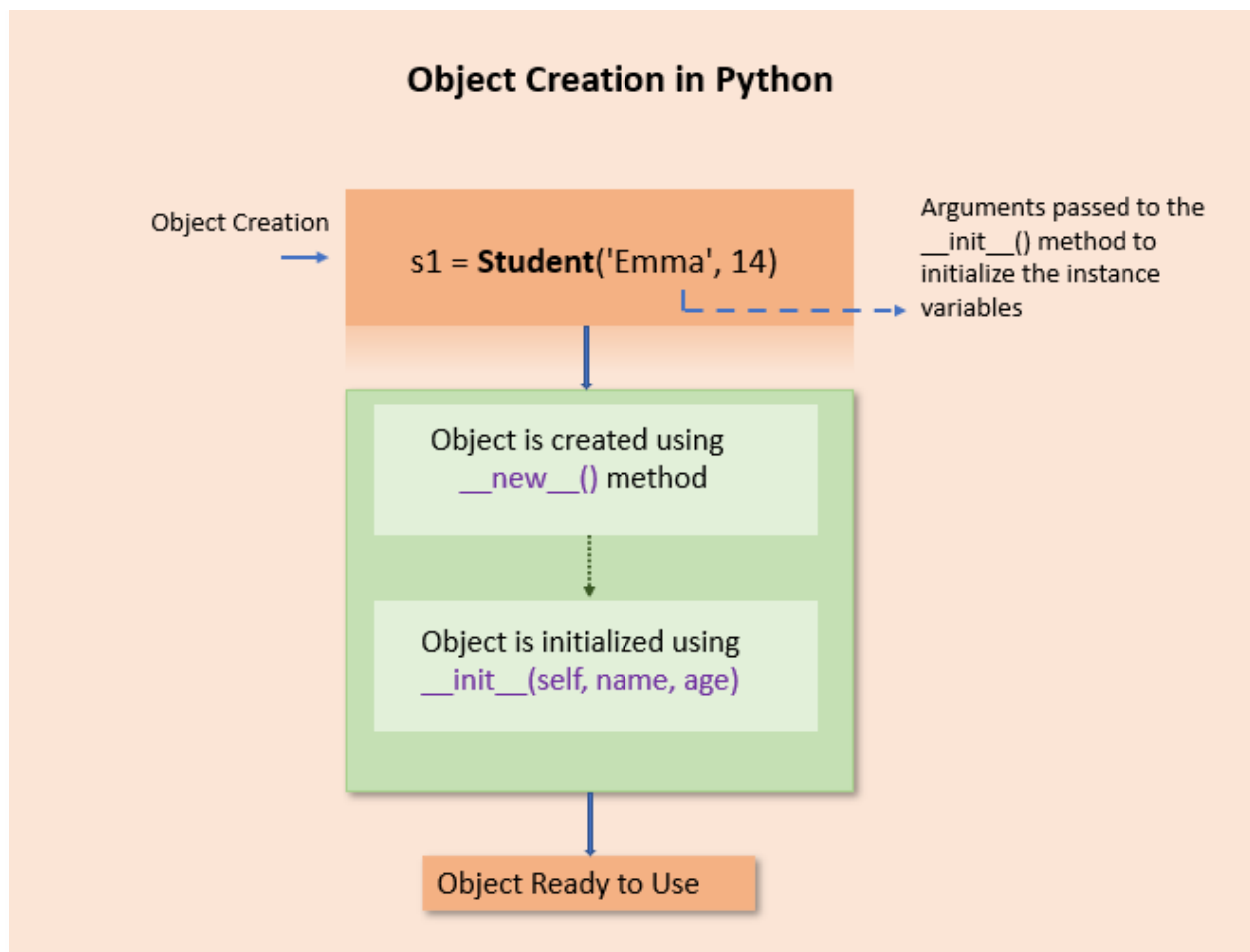
Output

Inside Constructor

All variables initialized

Hello, my name is Emma

- In the above example, an object s1 is created using the constructor
- While creating a Student object name is passed as an argument to the `__init__()` method to initialize the object.
- Similarly, various objects of the Student class can be created by passing different names as arguments.



Create an object in Python using a constructor

Types of Constructors

In Python, we have the following three types of constructors.

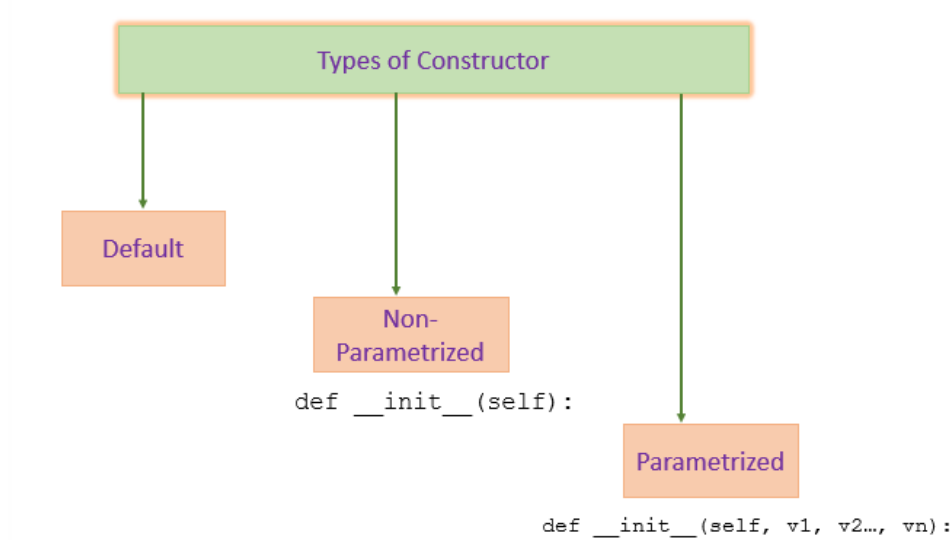
- Default Constructor
- Non-parametrized constructor
- Parameterized constructor

Default Constructor

Python will provide a default constructor if no constructor is defined. Python adds a default constructor when we do not include the constructor in the class or forget to declare it. It does not perform any task but initializes the objects. It is an empty constructor without a body.

If you do not implement any constructor in your class or forget to declare it, the Python inserts a default constructor into your code on your behalf. This constructor is known as the default constructor.

It does not perform any task but initializes the objects. It is an empty constructor without a body.



Note:

- The default constructor is not present in the source py file. It is inserted into the code during compilation if not exists. See the below image.
- If you implement your constructor, then the default constructor will not be added.

Example:

```
class Employee:
```

```
    def display(self):  
        print('Inside Display')
```

```
emp = Employee()  
emp.display()
```

Output

```
Inside Display
```

As you can see in the example, we do not have a constructor, but we can still create an object for the class because Python added the default constructor during a program compilation.

Non-Parametrized Constructor

A constructor without any arguments is called a non-parameterized constructor. This type of constructor is used to initialize each object with default values.

This constructor doesn't accept the arguments during object creation. Instead, it initializes every object with the same set of values.

```
class Company:
```

```
    # no-argument constructor
```

```
    def __init__(self):  
        self.name = "PYnative"  
        self.address = "ABC Street"
```

```
    # a method for printing data members
```

```
    def show(self):  
        print('Name:', self.name, 'Address:', self.address)
```

```
    # creating object of the class
```

```
    cmp = Company()
```

```
    # calling the instance method using the object
```

```
    cmp.show()
```

Output

Name: PYNative Address: ABC Street

As you can see in the example, we do not send any argument to a constructor while creating an object.

Parameterized Constructor

A constructor with defined parameters or arguments is called a parameterized constructor. We can pass different values to each object at the time of creation using a parameterized constructor.

The first parameter to constructor is `self` that is a reference to the being constructed, and the rest of the arguments are provided by the programmer. A parameterized constructor can have any number of arguments.

For example, consider a company that contains thousands of employees. In this case, while creating each employee object, we need to pass a different name, age, and salary. In such cases, use the parameterized constructor.

Example:

```
class Employee:
    # parameterized constructor
    def __init__(self, name, age, salary):
        self.name = name
        self.age = age
        self.salary = salary

    # display object
    def show(self):
        print(self.name, self.age, self.salary)

# creating object of the Employee class
emma = Employee('Emma', 23, 7500)
emma.show()

kelly = Employee('Kelly', 25, 8500)
kelly.show()
```

Output

Emma 23 7500

Kelly 25 8500

In the above example, we define a parameterized constructor which takes three parameters.

Constructor With Default Values

Python allows us to define a constructor with default values. The default value will be used if we do not pass arguments to the constructor at the time of object creation.

The following example shows how to use the default values with the constructor.

Example

```
class Student:
    # constructor with default values age and classroom
    def __init__(self, name, age=12, classroom=7):
        self.name = name
        self.age = age
        self.classroom = classroom

    # display Student
    def show(self):
        print(self.name, self.age, self.classroom)

# creating object of the Student class
emma = Student('Emma')
emma.show()

kelly = Student('Kelly', 13)
kelly.show()
```

Output

Emma 12 7

Kelly 13 7

As you can see, we didn't pass the age and classroom value at the time of object creation, so default values are used.

Self Keyword in Python

As you all know, the class contains instance variables and methods. Whenever we define instance methods for a class, we use self as the first parameter. Using self, we can **access the instance variable and instance method** of the object.

The first argument self refers to the current object.

Whenever we call an instance method through an object, the Python compiler implicitly passes object reference as the first argument commonly known as self.

It is not mandatory to name the first parameter as a self. We can give any name whatever we like, but it has to be the first parameter of an instance method.

Example

```
class Student:
    # constructor
    def __init__(self, name, age):
        self.name = name
        self.age = age

    # self points to the current object
    def show(self):
        # access instance variable using self
        print(self.name, self.age)

# creating first object
emma = Student('Emma', 12)
emma.show()

# creating Second object
kelly = Student('Kelly', 13)
kelly.show()
```

Output

Emma 12

Constructor Overloading

Constructor overloading is a concept of having more than one constructor with a different parameters list in such a way so that each constructor can perform different tasks.

For example, we can create a three constructor which accepts a different set of parameters

Python does not support constructor overloading. If we define multiple constructors then, the interpreter will consider only the last constructor and throws an error if the sequence of the arguments doesn't match as per the last constructor. The following example shows the same.

Example

```
class Student:
    # one argument constructor
    def __init__(self, name):
        print("One arguments constructor")
        self.name = name

    # two argument constructor
    def __init__(self, name, age):
        print("Two arguments constructor")
        self.name = name
        self.age = age

# creating first object
emma = Student('Emma')

# creating Second object
kelly = Student('Kelly', 13)
```

Output

```
TypeError: __init__() missing 1 required positional argument: 'age'
```

- As you can see in the above example, we defined multiple constructors with different arguments.
- At the time of object creation, the interpreter executed the second constructor because Python always considers the last constructor.

- Internally, the object of the class will always call the last constructor, even if the class has multiple constructors.
- In the example when we called a constructor only with one argument, we got a type error.

Constructor Chaining

Constructors are used for instantiating an object. The task of the constructor is to assign value to data members when an object of the class is created.

Constructor chaining is the process of calling one constructor from another constructor. Constructor chaining is useful when you want to invoke multiple constructors, one after another, by initializing only one instance.

In Python, **constructor chaining is convenient when we are dealing with inheritance**. When an instance of a child class is initialized, the constructors of all the parent classes are first invoked and then, in the end, the constructor of the child class is invoked.

Using the `super()` method we can invoke the parent class constructor from a child class.

Example

```
class Vehicle:
    # Constructor of Vehicle
    def __init__(self, engine):
        print('Inside Vehicle Constructor')
        self.engine = engine

class Car(Vehicle):
    # Constructor of Car
    def __init__(self, engine, max_speed):
        super().__init__(engine)
        print('Inside Car Constructor')
        self.max_speed = max_speed

class Electric_Car(Car):
    # Constructor of Electric Car
    def __init__(self, engine, max_speed, km_range):
        super().__init__(engine, max_speed)
        print('Inside Electric Car Constructor')
        self.km_range = km_range

# Object of electric car
ev = Electric_Car('1500cc', 240, 750)
```

```
print(f'Engine={ev.engine}, Max Speed={ev.max_speed}, Km range={ev.km_range}')
```

Counting the Number of objects of a Class

The constructor executes when we create the object of the class. For every object, the constructor is called only once. So for counting the number of objects of a class, we can add a counter in the constructor, which increments by one after each object creation.

Example

```
class Employee:
    count = 0
    def __init__(self):
        Employee.count = Employee.count + 1

# creating objects
e1 = Employee()
e2 = Employee()
e2 = Employee()
print("The number of Employee:", Employee.count)
```

Output

The number of employee: 3

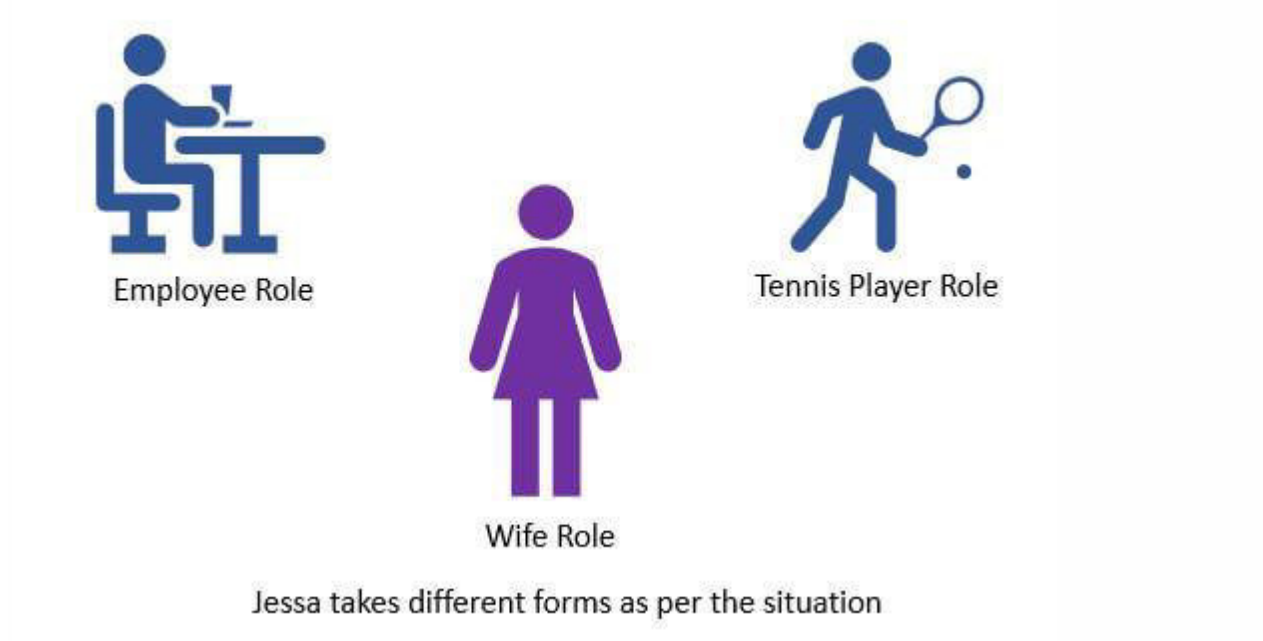
Polymorphism in Python

Object-Oriented Programming (OOP) has four essential characteristics: abstraction, encapsulation, inheritance, and polymorphism.

What is Polymorphism in Python?

Polymorphism in Python is the ability of an object to take many forms. In simple words, polymorphism allows us to perform the same action in many different ways.

For example, Jessa acts as an employee when she is at the office. However, when she is at home, she acts like a wife. Also, she represents herself differently in different places. Therefore, the same person takes different forms as per the situation.



In polymorphism, a method can **process objects differently depending on the class type or data type**. Let's see simple examples to understand it better.

Polymorphism in Built-in function `len()`

The built-in function `len()` calculates the length of an object depending upon its type. If an object is a string, it returns the count of characters, and If an object is a list, it returns the count of items in a list.

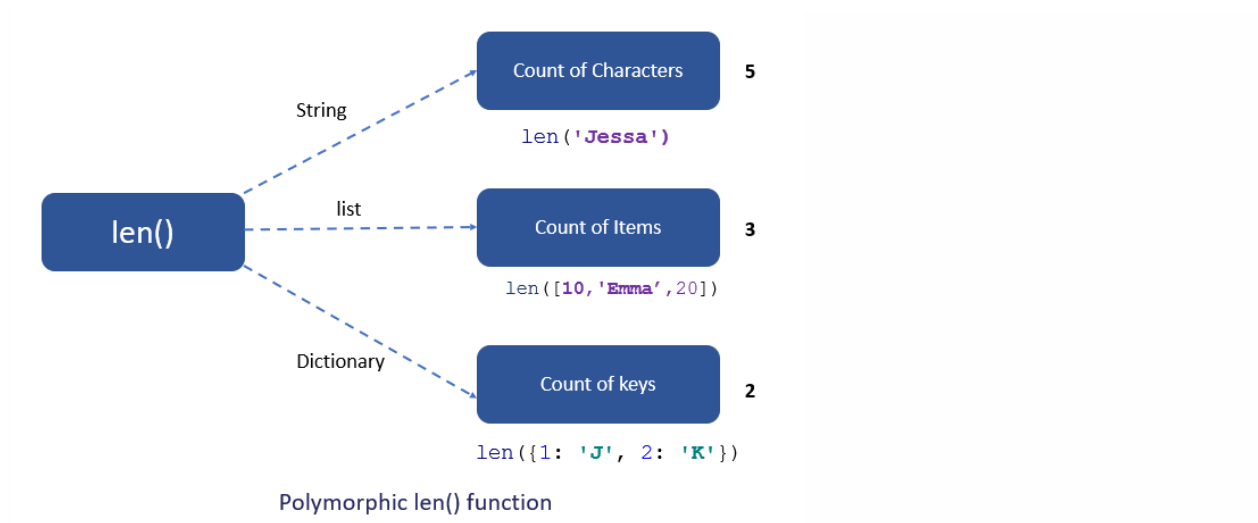
The `len()` method treats an object as per its class type.

Example:

```
students = ['Emma', 'Jessa', 'Kelly']
school = 'ABC School'

# calculate count
print(len(students))
print(len(school))
```

Output



Polymorphism With Inheritance

Polymorphism is mainly used with inheritance. In **inheritance**, child class inherits the attributes and methods of a parent class. The existing class is called a base class or parent class, and the new class is called a subclass or child class or derived class.

Using **method overriding** polymorphism allows us to define methods in the child class that have the same name as the methods in the parent class. This **process of re-implementing the inherited method in the child class** is known as Method Overriding.

Advantage of method overriding

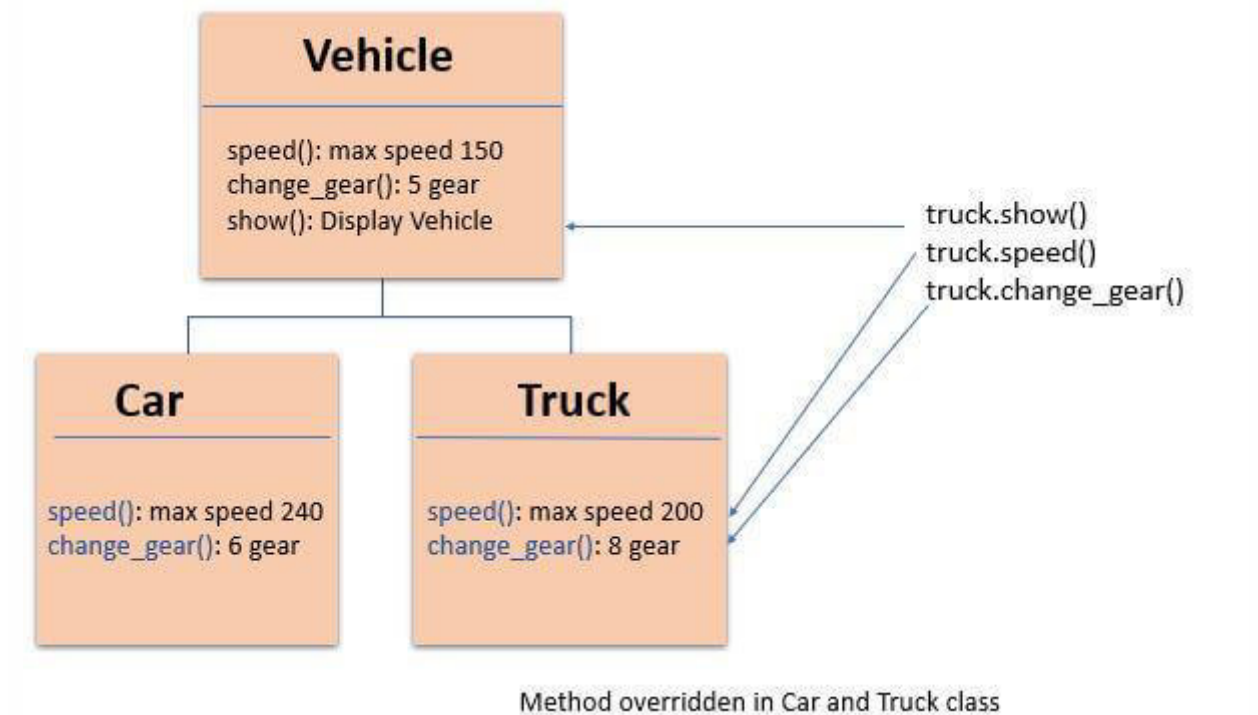
- It is effective when we want to extend the functionality by altering the inherited method. Or the method inherited from the parent class doesn't fulfill the need of a child class, so we need to re-implement the same method in the child class in a different way.
- Method overriding is useful when a parent class has multiple child classes, and one of that child class wants to redefine the method. The other child classes can use the parent class method. Due to this, we don't need to modify the parent class code.

In polymorphism, **Python first checks the object's class type and executes the appropriate method** when we call the method. For example, If you create the Car object, then Python calls the `speed()` method from a Car class.

Let's see how it works with the help of an example.

Example: Method Overriding

In this example, we have a vehicle class as a parent and a 'Car' and 'Truck' as its sub-class. But each vehicle can have a different seating capacity, speed, etc., so we can have the same instance method name in each class but with a different implementation. Using this code can be extended and easily maintained over time.



```
class Vehicle:
```

```
    def __init__(self, name, color, price):
        self.name = name
        self.color = color
        self.price = price
```

```
    def show(self):
        print('Details:', self.name, self.color, self.price)
```

```
    def max_speed(self):
        print('Vehicle max speed is 150')
```

```
    def change_gear(self):
        print('Vehicle change 6 gear')
```

```
# inherit from vehicle class
```

```
class Car(Vehicle):
    def max_speed(self):
        print('Car max speed is 240')

    def change_gear(self):
        print('Car change 7 gear')

# Car Object
car = Car('Car x1', 'Red', 20000)
car.show()
# calls methods from Car class
car.max_speed()
car.change_gear()

# Vehicle Object
vehicle = Vehicle('Truck x1', 'white', 75000)
vehicle.show()
# calls method from a Vehicle class
vehicle.max_speed()
vehicle.change_gear()
```

Output:

Details: Car x1 Red 20000

Car max speed is 240

Car change 7 gear

Details: Truck x1 white 75000

Vehicle max speed is 150

Vehicle change 6 gear

As you can see, due to polymorphism, the Python interpreter recognizes that the `max_speed()` and `change_gear()` methods are overridden for the car object. So, it uses the one defined in the child class (Car)

On the other hand, the `show()` method isn't overridden in the Car class, so it is used from the Vehicle class.

Overrride Built-in Functions

In Python, we can change the default behavior of the built-in functions. For example, we can change or extend the built-in functions such as `len()`, `abs()`, or `divmod()` by redefining them in our class. Let's see the example.

Example

In this example, we will redefine the function `len()`

```
class Shopping:
    def __init__(self, basket, buyer):
        self.basket = list(basket)
        self.buyer = buyer

    def __len__(self):
        print('Redefine length')
        count = len(self.basket)
        # count total items in a different way
        # pair of shoes and shir+pant
        return count * 2

shopping = Shopping(['Shoes', 'dress'], 'Jessa')
print(len(shopping))
```

Output

Redefine length

4

Polymorphism In Class methods

Polymorphism with class methods is useful when we group different objects having the same method. we can add them to a list or a tuple, and we don't need to check the object type before calling their methods. Instead, Python will check object type at runtime and call the correct method. Thus, we can call the methods without being concerned about which class type each object is. We assume that these methods exist in each class.

Python allows different classes to have methods with the same name.

- Let's design a different class in the same way by adding the same methods in two or more classes.
- Next, create an object of each class
- Next, add all objects in a tuple.
- In the end, iterate the tuple using a for loop and call methods of a object without checking its class.

Example

In the below example, fuel_type() and max_speed() are the instance methods created in both classes.

```
class Ferrari:
    def fuel_type(self):
        print("Petrol")

    def max_speed(self):
        print("Max speed 350")

class BMW:
    def fuel_type(self):
        print("Diesel")

    def max_speed(self):
        print("Max speed is 240")

ferrari = Ferrari()
bmw = BMW()

# iterate objects of same type
for car in (ferrari, bmw):
    # call methods without checking class of object
    car.fuel_type()
    car.max_speed()
```

Output

Petrol

Max speed 350

Diesel

Max speed is 240

As you can see, we have created two classes Ferrari and BMW. They have the same instance method names `fuel_type()` and `max_speed()`. However, we have not linked both the classes nor have we used inheritance.

We packed two different objects into a tuple and iterate through it using a `car` variable. It is possible due to polymorphism because we have added the same method in both classes Python first checks the object's class type and executes the method present in its class.

Polymorphism with Function and Objects

We can create polymorphism with a function that can take any object as a parameter and execute its method without checking its class type. Using this, we can call object actions using the same function instead of repeating method calls.

Example

```
class Ferrari:
    def fuel_type(self):
        print("Petrol")

    def max_speed(self):
        print("Max speed 350")

class BMW:
    def fuel_type(self):
        print("Diesel")

    def max_speed(self):
        print("Max speed is 240")

# normal function
def car_details(obj):
    obj.fuel_type()
    obj.max_speed()

ferrari = Ferrari()
bmw = BMW()

car_details(ferrari)
car_details(bmw)
```

Output

Petrol

Max speed 350

Diesel

Max speed is 240

Polymorphism In Built-in Methods

The word polymorphism is taken from the Greek words poly (many) and morphism (forms). It means a **method can process objects differently depending on the class type or data type.**

The built-in function `reversed(obj)` returns the iterable by reversing the given object. For example, if you pass a string to it, it will reverse it. But if you pass a list of strings to it, it will return the iterable by reversing the order of elements (it will not reverse the individual string).

Let us see how a built-in method process objects having different data types.

Example:

```
students = ['Emma', 'Jessa', 'Kelly']
school = 'ABC School'

print('Reverse string')
for i in reversed('PYnative'):
    print(i, end=' ')

print("\nReverse list")
for i in reversed(['Emma', 'Jessa', 'Kelly']):
    print(i, end=' ')
```

Output:

Reverse string

e v i t a n Y P

Reverse list

Kelly Jessa Emma

Method Overloading

The process of calling the same method with different parameters is known as method overloading. Python does not support method overloading. Python considers only the latest defined method even if you overload the method. Python will raise a `TypeError` if you overload the method.

Example

```
def addition(a, b):  
    c = a + b  
    print(c)  
  
def addition(a, b, c):  
    d = a + b + c  
    print(d)  
  
# the below line shows an error  
# addition(4, 5)  
  
# This line will call the second product method  
addition(3, 7, 5)
```

To overcome the above problem, we can use different ways to achieve the method overloading. In Python, to overload the class method, we need to write the method's logic so that different code executes inside the function depending on the parameter passes.

For example, the built-in function `range()` takes three parameters and produce different result depending upon the number of parameters passed to it.

Example:

```
for i in range(5): print(i, end=', '  
print()  
for i in range(5, 10): print(i, end=', '  
print()
```

```
for i in range(2, 12, 2): print(i, end=', ')
```

Output:

```
0, 1, 2, 3, 4,  
5, 6, 7, 8, 9,  
2, 4, 6, 8, 10,
```

Let's assume we have an `area()` method to calculate the area of a square and rectangle. The method will calculate the area depending upon the number of parameters passed to it.

- If one parameter is passed, then the area of a square is calculated
- If two parameters are passed, then the area of a rectangle is calculated.

Example: User-defined polymorphic method

```
class Shape:  
    # function with two default parameters  
    def area(self, a, b=0):  
        if b > 0:  
            print('Area of Rectangle is:', a * b)  
        else:  
            print('Area of Square is:', a ** 2)  
  
square = Shape()  
square.area(5)  
  
rectangle = Shape()  
rectangle.area(5, 3)
```

Output:

```
Area of Square is: 25  
  
Area of Rectangle is: 15
```

Operator Overloading in Python

Operator overloading means changing the default behavior of an operator depending on the operands (values) that we use. In other words, we can use the same operator for multiple purposes.

For example, the + operator will perform an arithmetic addition operation when used with numbers. Likewise, it will perform concatenation when used with strings.

The operator + is used to carry out different operations for distinct data types. This is one of the most simple occurrences of polymorphism in Python.

Example:

```
# add 2 numbers
print(100 + 200)

# concatenate two strings
print('Jess' + 'Roy')

# merger two list
print([10, 20, 30] + ['jessa', 'emma', 'kelly'])
```

Output:

```
300

JessRoy

[10, 20, 30, 'jessa', 'emma', 'kelly']
```

Overloading + operator for custom objects

Suppose we have two objects, and we want to add these two objects with a binary + operator. However, it will throw an error if we perform addition because the compiler doesn't add two objects. See the following example for more details.

Example:

```
class Book:
    def __init__(self, pages):
        self.pages = pages

# creating two objects
```

```
b1 = Book(400)
b2 = Book(300)

# add two objects
print(b1 + b2)
```

Output

```
TypeError: unsupported operand type(s) for +: 'Book' and 'Book'
```

We can overload `+` operator to work with custom objects also. Python provides some special or magic function that is automatically invoked when associated with that particular operator.

For example, when we use the `+` operator, the magic method `__add__()` is automatically invoked. Internally `+` operator is implemented by using `__add__()` method. We have to override this method in our class if you want to add two custom objects.

Example:

```
class Book:
    def __init__(self, pages):
        self.pages = pages

    # Overloading + operator with magic method
    def __add__(self, other):
        return self.pages + other.pages

b1 = Book(400)
b2 = Book(300)
print("Total number of pages: ", b1 + b2)
```

Output

```
Total number of pages: 700
```

Overloading the `*` Operator

The `*` operator is used to perform the multiplication. Let's see how to overload it to calculate the salary of an employee for a specific period. Internally `*` operator is implemented by using the `__mul__()` method.

Example:

```
class Employee:
    def __init__(self, name, salary):
        self.name = name
        self.salary = salary

    def __mul__(self, timesheet):
        print('Worked for', timesheet.days, 'days')
        # calculate salary
        return self.salary * timesheet.days

class TimeSheet:
    def __init__(self, name, days):
        self.name = name
        self.days = days

emp = Employee("Jessa", 800)
timesheet = TimeSheet("Jessa", 50)
print("salary is: ", emp * timesheet)
```

Output

Worked for 50 days

salary is: 40000

Encapsulation in Python

Encapsulation is one of the fundamental concepts in object-oriented programming (OOP), including abstraction, inheritance, and polymorphism. This lesson will cover what encapsulation is and how to implement it in Python.

What is Encapsulation in Python?

Encapsulation in Python describes the concept of **bundling data and methods within a single unit**. So, for example, when you create a class, it means you are implementing encapsulation. A class is an example of encapsulation as it binds all the data members (instance variables) and methods into a single unit.

Example:

In this example, we create an Employee class by defining employee attributes such as name and salary as an instance variable and implementing behavior using work() and show() instance methods.

```
class Employee:
    # constructor
    def __init__(self, name, salary, project):
        # data members
        self.name = name
        self.salary = salary
        self.project = project

    # method
    # to display employee's details
    def show(self):
        # accessing public data member
        print("Name: ", self.name, 'Salary:', self.salary)

    # method
    def work(self):
        print(self.name, 'is working on', self.project)

# creating object of a class
emp = Employee('Jessa', 8000, 'NLP')

# calling public method of the class
emp.show()
emp.work()
```

Output:

```
Name:  Jessa Salary: 8000
```

```
Jessa is working on NLP
```

Using encapsulation, we can hide an object's internal representation from the outside. This is called information hiding.

Also, encapsulation allows us to restrict accessing variables and methods directly and prevent accidental data modification by creating private data members and methods within a class.

Encapsulation is a way to can restrict access to methods and variables from outside of class. Whenever we are working with the class and dealing with sensitive data, providing access to all variables used within the class is not a good choice.

For example, Suppose you have an attribute that is not visible from the outside of an object and bundle it with methods that provide read or write access. In that case, you can hide specific information and control access to the object's internal state. Encapsulation offers a way for us to access the required variable without providing the program full-fledged access to all variables of a class. This mechanism is used to protect the data of an object from other objects.

Access Modifiers in Python

Encapsulation can be achieved by declaring the data members and methods of a class either as private or protected. But In Python, we don't have direct access modifiers like public, private, and protected. We can achieve this by using single **underscore** and **double underscores**.

Access modifiers limit access to the variables and methods of a class. Python provides three types of access modifiers private, public, and protected.

- **Public Member:** Accessible anywhere from outside class.
- **Private Member:** Accessible within the class
- **Protected Member:** Accessible within the class and its sub-classes

Public Member

Public data members are accessible within and outside of a class. All member variables of the class are by default public.

Example:

```
class Employee:
    # constructor
    def __init__(self, name, salary):
        # public data members
        self.name = name
        self.salary = salary

    # public instance methods
    def show(self):
        # accessing public data member
        print("Name: ", self.name, 'Salary:', self.salary)
```

```
# creating object of a class
emp = Employee('Jessa', 10000)

# accessing public data members
print("Name: ", emp.name, 'Salary:', emp.salary)

# calling public method of the class
emp.show()
```

Output

Name: Jessa Salary: 10000

Name: Jessa Salary: 10000

Private Member

We can protect variables in the class by marking them private. To define a private variable add two underscores as a prefix at the start of a variable name.

Private members are accessible only within the class, and we can't access them directly from the class objects.

Example:

```
class Employee:
    # constructor
    def __init__(self, name, salary):
        # public data member
        self.name = name
        # private member
        self.__salary = salary

# creating object of a class
emp = Employee('Jessa', 10000)

# accessing private data members
print('Salary:', emp.__salary)
```

Output

AttributeError: 'Employee' object has no attribute '__salary'

In the above example, the salary is a private variable. As you know, we can't access the private variable from the outside of that class.

We can access private members from outside of a class using the following two approaches

- Create public method to access private members
- Use name mangling

Let's see each one by one

Public method to access private members

Example: Access Private member outside of a class using an instance method

```
class Employee:
    # constructor
    def __init__(self, name, salary):
        # public data member
        self.name = name
        # private member
        self.__salary = salary

    # public instance methods
    def show(self):
        # private members are accessible from a class
        print("Name: ", self.name, 'Salary:', self.__salary)

# creating object of a class
emp = Employee('Jessa', 10000)

# calling public method of the class
emp.show()
```

Output:

Name: Jessa Salary: 10000

Name Mangling to access private members

We can directly access private and protected variables from outside of a class through name mangling. The name mangling is created on an identifier by adding two leading underscores and one trailing underscore, like this `_classname__dataMember`, where `classname` is the current class, and `data member` is the private variable name.

Example: Access private member

```
class Employee:
    # constructor
    def __init__(self, name, salary):
        # public data member
        self.name = name
        # private member
        self.__salary = salary

# creating object of a class
emp = Employee('Jessa', 10000)

print('Name:', emp.name)
# direct access to private member using name mangling
print('Salary:', emp._Employee__salary)
```

Run

Output

Name: Jessa

Salary: 10000

Protected Member

Protected members are accessible within the class and also available to its sub-classes. To define a protected member, prefix the member name with a single underscore `_`.

Protected data members are used when you implement inheritance and want to allow data members access to only child classes.

Example: Protected member in inheritance.

```
# base class
class Company:
    def __init__(self):
        # Protected member
        self._project = "NLP"

# child class
class Employee(Company):
    def __init__(self, name):
        self.name = name
        Company.__init__(self)

    def show(self):
        print("Employee name :", self.name)
        # Accessing protected member in child class
        print("Working on project :", self._project)

c = Employee("Jessa")
c.show()

# Direct access protected data member
print('Project:', c._project)
```

Output

```
Employee name : Jessa

Working on project : NLP

Project: NLP
```

Getters and Setters in Python

To implement proper encapsulation in Python, we need to use setters and getters. The primary purpose of using getters and setters in object-oriented programs is to ensure data encapsulation. Use the getter method to access data members and the setter methods to modify the data members.

In Python, private variables are not hidden fields like in other programming languages. The getters and setters methods are often used when:

- When we want to avoid direct access to private variables

- To add validation logic for setting a value
- **Example**

```
• class Student:
•     def __init__(self, name, age):
•         # private member
•         self.name = name
•         self.__age = age
•
•         # getter method
•         def get_age(self):
•             return self.__age
•
•         # setter method
•         def set_age(self, age):
•             self.__age = age
•
• stud = Student('Jessa', 14)
•
• # retrieving age using getter
• print('Name:', stud.name, stud.get_age())
•
• # changing age using setter
• stud.set_age(16)
•
• # retrieving age using getter
• print('Name:', stud.name, stud.get_age())
```

- **Output**

- Name: Jessa 14
- Name: Jessa 16

- Let's take another example that shows how to use encapsulation to implement information hiding and apply additional validation before changing the values of your object attributes (data member).
- **Example:** Information Hiding and conditional logic for setting an object attributes

```
• class Student:
```

```
• def __init__(self, name, roll_no, age):
•     # private member
•     self.name = name
•     # private members to restrict access
•     # avoid direct data modification
•     self.__roll_no = roll_no
•     self.__age = age
•
• def show(self):
•     print('Student Details:', self.name, self.__roll_no)
•
•     # getter methods
• def get_roll_no(self):
•     return self.__roll_no
•
•     # setter method to modify data member
•     # condition to allow data modification with rules
• def set_roll_no(self, number):
•     if number > 50:
•         print('Invalid roll no. Please set correct roll number')
•     else:
•         self.__roll_no = number
•
• jessa = Student('Jessa', 10, 15)
•
• # before Modify
• jessa.show()
• # changing roll number using setter
• jessa.set_roll_no(120)
•
•
• jessa.set_roll_no(25)
• jessa.show()
```

• **Output:**

- Student Details: Jessa 10
- Invalid roll no. Please set correct roll number
-
- Student Details: Jessa 25

Advantages of Encapsulation

- **Security:** The main advantage of using encapsulation is the security of the data. Encapsulation protects an object from unauthorized access. It allows private and protected access levels to prevent accidental data modification.
- **Data Hiding:** The user would not be knowing what is going on behind the scene. They would only be knowing that to modify a data member, call the setter method. To read a data member, call the getter method. What these setter and getter methods are doing is hidden from them.
- **Simplicity:** It simplifies the maintenance of the application by keeping classes separated and preventing them from tightly coupling with each other.
- **Aesthetics:** Bundling data and methods within a class makes code more readable and maintainable

Data Abstraction in Python

Data Abstraction in Python:

Data Abstraction in Python is the process of hiding the real implementation of an application from the user and emphasizing only on usage of it. Basically, Abstraction focuses on hiding the internal implementations of a process or method from the user. In this way, the user knows what he is doing but not how the work is being done.

Why Do We Need Abstraction?

Through the process of abstraction in Python, a programmer can hide all the irrelevant data of an application to reduce complexity and increase efficiency.

Syntax:

```
from abc import ABC
```

```
Class ClassName(ABC):
```

In Python abstraction can be achieved using abstract class and it's methods.

Abstract Class in Python

- A class containing one or more abstract methods is called an abstract class.
- As a property, abstract classes can have any number of abstract methods coexisting with any number of other methods.

- Abstract methods do not contain any implementation. Instead, all the implementations can be defined in the methods of sub-classes that inherit the abstract class.
- We have already discussed the syntax of abstract class above, let's understand this by taking examples.

Data abstraction

Code #1:

```
#python Program
```

```
#Rishikesh
```

```
#Data Abstraction
```

```
#oops in Python
```

```
from abc import ABC
```

```
class llgm(ABC): #abstract classdef calculate_area(self): #abstract methodpass
```

```
    pass
```

```
class Square(llgm):
```

```
    length = 5
```

```
    def Area(self):
```

```
        return self.length * self.length
```

```
class Circle(llgm):
```

```
    radius =4
```

```
    def Area(self):
```

```
        return 3.14 * self.radius * self.radius
```

```
sq = Square() #object created for the class 'Square'
```

```
cir = Circle() #object created for the class 'Circle'
```

```
print("Area of a Square:", sq.Area()) #call to 'calculate_area' method defined inside the class 'Square'
```

```
print("Area of a circle:", cir.Area()) #call to 'calculate_area' method defined inside the class 'Circle'.
```

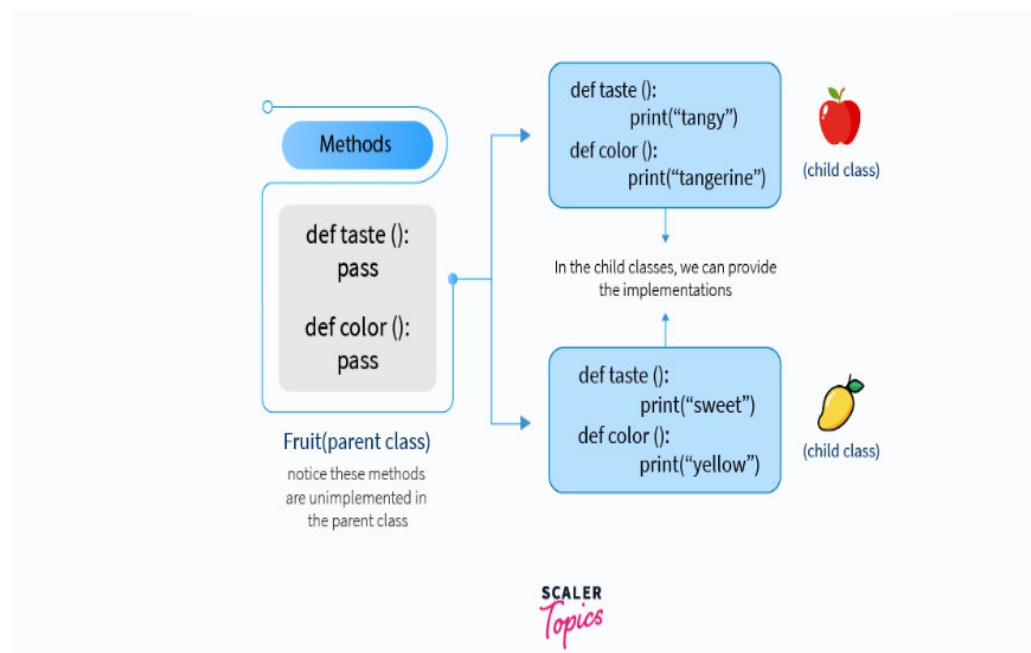
Output:

Area of a Square: 25

Area of a circle: 50.24

An abstract class can have both a normal method and an abstract method

An abstract class cannot be instantiated, (we cannot create objects for the abstract class).



Concrete Methods in Abstract Base Class in Python

We now know that we use abstract classes as a template for other similarly characterized classes. Using this, we can define a structure, but do not need to provide complete implementation for every method, such as:

```
from abc import ABC, abstractmethod
class MyClass(ABC):
    @abstractmethod
    def mymethod(self):
        #empty body
        pass
```

The methods where the implementation may vary for any other subclass are defined as abstract methods and need to be given an implementation in the subclass definition.

On the other hand, there are methods that have the same implementation for all subclasses as well. There are characteristics that exhibit the properties of the abstract class and so, must be implemented in the abstract class itself. Otherwise, it will lead to repetitive code in all the inherited classes. These methods are called concrete methods.

```
from abc import ABC, abstractmethod

class Parent(ABC):
    #common function
    def common_fn(self):
        print('In the common method of Parent')
    @abstractmethod
    def abs_fn(self): #is supposed to have different implementation in child classes
        pass

class Child1(Parent):
    def abs_fn(self):
        print('In the abstract method of Child1')

class Child2(Parent):
    def abs_fn(self):
        print('In the abstract method of Child2')
```

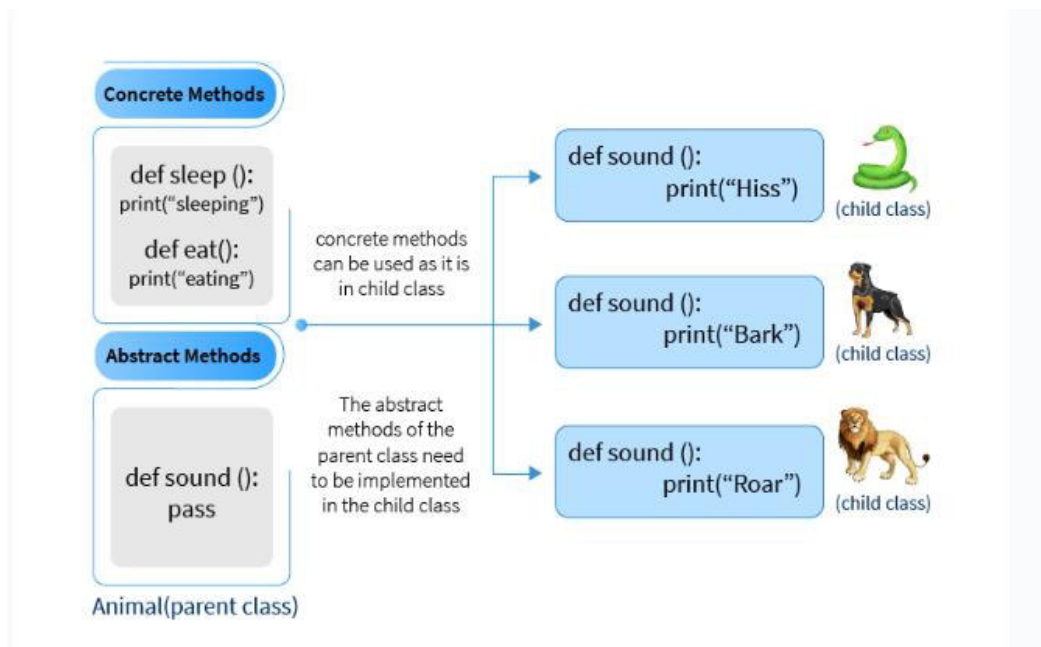
An abstract class can have both abstract methods and concrete methods.

We can now access the concrete method of the abstract class by instantiating an object of the child class. We can also access the abstract methods of the child classes with it. Interesting points to keep in mind are:

- We always need to provide an implementation of the abstract method in the child class even when implementation is given in the abstract class.
- A subclass must implement all abstract methods that are defined in the parent class otherwise it results in an error.

Examples of Data Abstraction

Let us take some examples to understand the working of abstract classes in Python. Consider the Animal parent class and other child classes derived from it.



```
from abc import ABC, abstractmethod

class Animal(ABC):

    #concrete method
    def sleep(self):
        print("I am going to sleep in a while")

    @abstractmethod
    def sound(self):
        print("This function is for defining the sound by any animal")
        pass

class Snake(Animal):
    def sound(self):
        print("I can hiss")

class Dog(Animal):
    def sound(self):
        print("I can bark")

class Lion(Animal):
    def sound(self):
        print("I can roar")
```

```
class Cat(Animal):
    def sound(self):
        print("I can meow")
```

Our abstract base class has a concrete method `sleep()` that will be the same for all the child classes. So, we do not define it as an abstract method, thus saving us from code repetition. On the other hand, the sounds that animals make are all different. For that purpose, we defined the `sound()` method as an abstract method. We then implement it in all child classes.

Now, when we instantiate the child class object, we can access both the concrete and the abstract methods.

```
c = Cat()
c.sleep()
c.sound()

c = Snake()
c.sound()
```

This will give the output as:

```
I am going to sleep in a while
I can meow
I can hiss
```

Now, if we want to access the `sound()` function of the base class itself, we can use the object of the child class, but we will have to invoke it through `super()`.

```
class Rabbit(Animal):
    def sound(self):
        super().sound()
        print("I can squeak")

c = Rabbit()
c.sound()
```

This will produce the following output:

```
This function is for defining the sound by any animal
I can squeak
```

If we do not provide any implementation of an abstract method in the derived child class, an error is produced. Notice, even when we have given implementation of the `sound()` method in the base class, not providing an implementation in the child class will produce an error.

```
class Deer(Animal):
    def sound(self):
        pass

c = Deer()
c.sound()
c.sleep()
```

This will produce the following error:

```
Traceback (most recent call last):
  File "F:/Python/Test/Parent.py", line 38, in <module>
    c = Deer()
TypeError: Can't instantiate abstract class Deer with abstract methods sound
```

NOTE: Had there been more than one abstract method in the base class, all of them are required to be implemented in the child classes, otherwise, an error is produced.

Method Overloading:

Two or more methods have the same name but different numbers of parameters or different types of parameters, or both. These methods are called overloaded methods and this is called method **overloading**.

Like other languages (for example, method overloading in C++) do, python does not support method overloading by default. But there are different ways to achieve method overloading in Python.

The problem with method overloading in Python is that we may overload the methods but can only use the latest defined method.

```
# First product method.
# Takes two argument and print their
# product
```

```
def product(a, b):
    p = a * b
    print(p)
```

```
# Second product method
# Takes three argument and print their
# product
```

```
def product(a, b, c):
    p = a * b * c
    print(p)
```

Uncommenting the below line shows an error

```
# product(4, 5)
```

```
# This line will call the second product method  
product(4, 5, 5)
```

Output

100

In the above code, we have defined two product methods we can only use the second product method, as python does not support method overloading. We may define many methods of the same name and different arguments, but we can only use the latest defined method. Calling the other method will produce an error. Like here calling **product(4,5)** will produce an error as the latest defined product method takes three arguments.

Thus, to overcome the above problem we can use different ways to achieve the method overloading.

Method 1 (Not The Most Efficient Method):

We can use the arguments to make the same function work differently i.e. as per the arguments.

```
# Function to take multiple arguments  
  
def add(datatype, *args):  
    # if datatype is int  
    # initialize answer as 0  
    if datatype == 'int':  
        answer = 0  
    # if datatype is str  
    # initialize answer as "  
    if datatype == 'str':  
        answer = "  
    # Traverse through the arguments  
    for x in args:
```

```
# This will do addition if the
# arguments are int. Or concatenation
# if the arguments are str
answer = answer + x

print(answer)

# Integer
add('int', 5, 6)

# String
add('str', 'Hi ', 'Geeks')
```

Output

11

Hi Geeks

Method 2 (Not the efficient one):

We can achieve method overloading in python by user defined function using “**None**” keyword as default parameter.

Code explanation:

The first parameter of “add” method is set to None. This will give us the option to call it with or without a parameter.

When we pass arguments to the add method (Working):

The method checks if both the parameters are available or not.

As we have already given default parameter values as “None”, if any of the value is not passed it will remain “None”.

Using If-Else statements, we can achieve method overloading by checking each parameter as single statement.

Python3

```
# code

def add(a=None, b=None):

    # Checks if both parameters are available

    # if statement will be executed if only one parameter is available

    if a != None and b == None:

        print(a)

    # else will be executed if both are available and returns addition of two

    else:

        print(a+b)

    # two arguments are passed, returns addition of two

add(2, 3)

# only one argument is passed, returns a

add(2)
```

Output

5

2

The problem with above methods is that, it makes code more complex with multiple if/else statement and is not the desired way to achieve the method overloading.

Method 3 (Efficient One):

By Using Multiple Dispatch Decorator

Multiple Dispatch Decorator Can be installed by:

pip3 install multipledispatch

```
from multipledispatch import dispatch

# passing one parameter
@dispatch(int, int)
def product(first, second):
    result = first*second
    print(result)

# passing two parameters
@dispatch(int, int, int)
def product(first, second, third):
    result = first * second * third
    print(result)

# you can also pass data type of any value as per requirement
@dispatch(float, float, float)
def product(first, second, third):
    result = first * second * third
    print(result)

# calling product method with 2 arguments
product(2, 3) # this will give output of 6

# calling product method with 3 arguments but all int
product(2, 3, 2) # this will give output of 12

# calling product method with 3 arguments but all float
product(2.2, 3.4, 2.3) # this will give output of 17.985999999999997
```

Output:

12

17.985999999999997

Method Overriding

Method Overriding in Python is an OOPs concept closely related to inheritance. When a child class method overrides(or, provides its own implementation) the parent class method of the same name, parameters and return type, it is known as method overriding.

Method overriding is completely different from the concept of method overloading. Method overloading occurs when there are two functions with the same name but different parameters. And, method overloading is not directly supported in Python.

Parent class: The class being **inherited** is called the Parent or Superclass.

Child class : The class that **inherits** the properties and methods of the parent class is called the Child or Subclass.

Note:

Inheritance is the capability of one class to derive or inherit the properties from another class, hence promoting code reusability.

Key features of Method Overriding in Python

These are some of the key features and advantages of method overriding in Python --

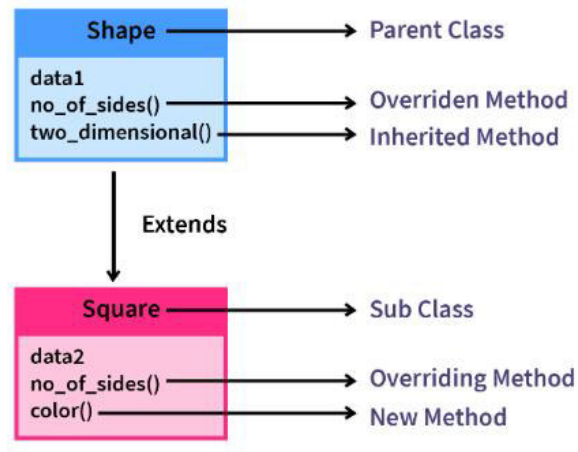
- Method Overriding is derived from the concept of object oriented programming
- Method Overriding allows us to change the implementation of a function in the child class which is defined in the parent class.
- Method Overriding is a part of the inheritance mechanism
- Method Overriding avoids duplication of code
- Method Overriding also enhances the code adding some additional properties.

Prerequisites for method overriding

There are certain prerequisites for method overriding in Python. They're discussed below --

1. Method overriding cannot be done within a class. So, we need to derive a child class from a parent class. Hence **Inheritance** is mandatory.
2. The method must have the **same name** as in the parent class
3. The method must have the **same number of parameters** as in the parent class.

Example of Method Overriding in Python:



So, in the above example we can see,

- **no_of_sides** is the **overriden** method in the Shape class, since the Square class method - **no_of_sides()** will be **overriding** it (adding it's own implementation).
- **two_dimensional** is **inherited** by the sub_class(hence inherited method)
- **color** is a **new method** added in the Square class.
- **data1** & **data2** are individual **properties** of the classes.

Let us code the above example and understand more deeply. Here we have our **parent class Shape** --

```
# Parent class
class Shape:
    # properties
    data1 = "abc"

    # function no_of_sides
    def no_of_sides(self):
        print("My sides need to be defined. I am from shape class.")

    # function two_dimensional
    def two_dimensional(self):
        print("I am a 2D object. I am from shape class")
```

Let's create a **child class - Square** which will extend the class Shape:

```
class Square (Shape):  
    data2 = "XYZ"  
  
    def no_of_sides (self):  
        print("I have 4 sides. I am from Square class")  
  
    def color(self):  
        print("I have teal color. I am from Square class.")
```

In the child class - Square, we have **overridden** the method **no_of_sides()** adding our own implementation.

So now, when we create an object of the Square class and call the method **no_of_sides()** we will see, it will override the already existing version of the parent class.

```
# Create an object of Square class  
sq = Square()  
# Override the no_of_sides of parent class  
sq.no_of_sides()  
# Will inherit this method from the parent class  
sq.two_dimensional()  
# It's own method color  
sq.color()
```

OUTPUT:

```
F:\Pycharm\venv\Scripts\python.exe F:/Pycharm/MethodOverridingExampe.py  
I have 4 sides. I am from Square class  
I am a 2D object. I am from shape class  
I have teal color. I am from Square class.
```

Not only that, we can also **override** the **data1 variable** present in the parent class(Shape) using the Child Class(Square) --

```
sq = Square()  
print("Old value of data1 = ", sq.data1)  
sq.data1 = "New value"  
print("The value of data1 in Shape class overridden by the Square class = ", sq.data1)
```

OUTPUT:

```
F:\Pycharm\venv\Scripts\python.exe F:/Pycharm/MethodOverridingExampe.py  
Old value of data1 = abc
```

The value of data1 in Shape class overridden by the Square class = New value

Complete code:

```
# Parent class
class Shape:
    # properties
    data1 = "abc"

    # function no_of_sides
    def no_of_sides(self):
        print("My sides need to be defined. I am from shape class.")

    # function two_dimensional
    def two_dimensional(self):
        print("I am a 2D object. I am from shape class")

# Sub-class
class Square(Shape):
    data2 = "xyz"

    def no_of_sides(self):
        print("I have 4 sides. I am from Square class")

    def color(self):
        print("I have teal color. I am from Square class.")

# Create an object of Square class
sq = Square()
# Override the no_of_sides of parent class
sq.no_of_sides()
# Will inherit this method from the parent class
sq.two_dimensional()
# It's own method - color
sq.color()
print("Old value of data1 = ", sq.data1)
# Override property of the Parent class
sq.data1 = "New value"
print("The value of data1 in Shape class overridden by the Square class = ", sq.data1)
```

OUTPUT:

```
I have 4 sides. I am from Square class
I am a 2D object. I am from shape class
I have teal color. I am from Square class.
```

```
Old value of data1 = abc  
The value of data1 in Shape class overridden by the Square class = New value
```

Method Resolution Order in Python

Method resolution order in Python is the order in which a python program resolves or searches for a class in case of multiple inheritance.

Important points to be noted --

- It plays a crucial role for programs using **multi-inheritance**.
- In multi-inheritance a class is derived from more than one base class, hence inheriting all properties of the base class
- In this approach -> the method or attributes are first searched in the base class. If it is not present then the searching moves a level up -- to the immediate parent class. And if again no luck, then the searching continues following the same search approach.
- It follows the depth-first search approach.

Now let us understand this through an example -- **Diamond Inheritance:**

Diamond Inheritance refers to an **ambiguity** that arises when two classes -- B and C inherit from a superclass A and class D inherits from both class B and C. Now, suppose there is a method “demo” which is an overridden method in one of Class B and Class C or both then the ambiguity arises which of the method “demo” Class D should inherit.

The problem is solved in a depth first search fashion.

Note:

Diamond Inheritance is a very good example of overriding in multiple inheritance in Python.

Let us take an example to understand this problem better --

Scenario 1:

Here we have a base class A, and B & C are the sub class, inheriting from A. Class D inherits from B & C, also getting the access to methods of class A. So, this is an example of a multi-level inheritance. Let us see the code:

```
class A:  
    def demo (self):  
        print(" From class A")  
  
class B(A):
```

```
def demo(self):
    print(" From class B")

class C(A):
    def demo(self):
        print(" From class C")

# To check MRO
class D(B,C):
    def check(self):
        print("This is an example showing Method Resolution order")

obj = D()
obj.demo()
```

In this code the order of execution will be : **class D -> class B -> class C -> class A**

So, the output will be:

```
F:\Pycharm\venv\Scripts\python.exe F:/Pycharm/MethodResolutionOrderExample.py
From class B
```

Since, the class D will find the method demo() instantly in class B so will execute class B's method and will not search in other classes.

Scenario 2:

Suppose, we pass the parameters to D in some other order -- like **class D(C,B)**

```
class A:
    def demo(self):
        print(" From class A")

class B(A):
    def demo(self):
        print(" From class B")

class C(A):
    def demo(self):
        print(" From class C")

# Passing parameter in other order:
class D(C,B):
```

```
def check(self):  
    print("This is an example showing Method Resolution order")  
  
obj = D()  
obj.demo()
```

Then MRO will resolve it in this order:

class D -> class C -> class B -> class A

So, since it finds the method in class C at the first chance, it executes the method in that class itself.

Output:

```
F:\Pycharm\venv\Scripts\python.exe F:/Pycharm/MethodResolutionOrderExample.py  
From class C
```

Scenario 3:

But, let us violate the structure as shown in the tree and pass in this order :
class D(A,B,C):

```
  A  
 / \  
B   C  
 \  
  D  
class D(A,B,C):  
    def check(self):  
        print("Disobeying the MRO")
```

It is a clear violation, because D cannot start finding the method from A and eventually go to B then C; Then definitely, we will get the error:

```
F:\Pycharm\venv\Scripts\python.exe F:/Pycharm/MethodResolutionOrderExample.py  
Traceback (most recent call last):  
  File "F:/Pycharm/MethodResolutionOrderExample.py", line 17, in <module>  
    class D(A,B,C):  
TypeError: Cannot create a consistent method resolution order (MRO) for bases A, B, C  
  
Process finished with exit code 1
```

Since, D cannot resolve the order, going straight to A and then coming to B & C. So here we get the `TypeError`. But we will not do the same if we pass the parameter in this fashion:

```
class D(B,C,A)
    def check(self):
        print("...")
```

Then we will get the intended output:

```
F:\Pycharm\venv\Scripts\python.exe F:/Pycharm/MethodResolutionOrderExample.py
From class B
```

So MRO is nothing, but the order in which a class searches for any method/attribute in its parent classes in a DFS fashion. And, the order of class we pass as parameters to the class is very important, otherwise it will result in error if violated.

Overriding With Multiple and Multilevel Inheritances

Inheritance:

It is the capability of one class to derive or inherit the properties from another class. It promotes reusability of code.

Note:

Inheritance is transitive in nature, i.e. if any class B inherits from another class A, then all the subclasses of B would automatically inherit from class A.

Multiple Inheritance

When a class is derived from one or more parent(base) class, inheriting all its properties, it is called the multiple inheritance. In the above image, D is inheriting from multiple parents(B & C).

Overriding in Multiple Inheritance

If the child class provides its own implementation to the methods/attributes while inheriting them from its parent/parents then it is known as Overriding in Multiple Inheritance.

Example:

The Diamond Problem we discussed above is a great example of Overriding in Multiple Inheritance.

Let us take one more example to further clarify our concept:

```
class Book:
    def subjectName(self):
        print(" This is a generic Book")

    def aboutMe(self):
        print("Hi I'm from book!")

class Physics (Book):
    def subjectName(self):
        print(" This is a Physics Book ")

class Chemistry (Book):
    def subjectName(self):
        print("This is a Chemistry Book")

class Science (Physics, Chemistry):
    def subjectName(self):
        print("This is a Science Book")

obj = Science()
obj.subjectName() #will execute the method from the class Science
obj.aboutMe() #will execute the method from the class Book
```

Here, when function call takes place, it goes in a depth first search fashion and provides the output.

```
F:\Pycharm\venv Scripts\python.exe F:
This is a Science Book
Hi I'm from book!
```

Multi-level Inheritance:

When we have a child and grandchild relationship in our code where we have different levels relationship, it is known as Multi-level Inheritance. In the above image, class 3 is inheriting from class 2 which in turn is inheriting from class 1. So 3 is the Grandchildren, 2 is the Parent and 1 is the GrandParent.

Overriding in Multi-level Inheritance:

When we override the methods or attributes of the Parent class or GrandParent class through the child class(class 3 here), then it is known as Overriding in Multi-level Inheritance. In this example above, if class 3 provides it's own implementation for class 2 and class 1 then it is Overriding in Multi-level Inheritance.

Example:

```
class GrandFather:
    def home (self):
        print("This is Mr. A's House")

    def age(self):
        print("Hi I'm 70 years old!")

class Father(Grandfather):
    def age(self):
        print("Hi I'm 45 years old!")

class Son(Father):
    def age(self):
        print("Hi I'm 20 years old!")

obj = Son()
obj.home()
obj.age()
```

Here the father inherits the method (home) from his father.

And the Son inherits "home" from his father's father(Grandfather).

Although, they provide separate implementation for their ages hence overriding this method.

OUTPUT:

```
F:\Pycharm\venv Scripts\python.exe F:/Pycharm/MultilevelInheritance.py
This is Mr. A's House
Hi I'm 20 years old!

Process finished with exit code 0
```

Hence we see, the home() method is inherited but the age method is overridden by the class Son.

Data Hiding:

What is Data Hiding?

Data hiding is a concept which underlines the hiding of data or information from the user. It is one of the key aspects of Object-Oriented programming strategies. It includes object details such as

data members, internal work. Data hiding excludes full data entry to class members and defends object integrity by preventing unintended changes.

Data hiding also minimizes system complexity for increase robustness by limiting interdependencies between software requirements. Data hiding is also known as information hiding. In class, if we declare the data members as private so that no other class can access the data members, then it is a process of hiding data.

Data Hiding in Python:

The Python document introduces Data Hiding as isolating the user from a part of program implementation. Some objects in the module are kept internal, unseen, and unreachable to the user. Modules in the program are easy enough to understand how to use the application, but the client cannot know how the application functions. Thus, data hiding imparts security, along with discarding dependency. Data hiding in Python is the technique to defend access to specific users in the application.

Python is applied in every technical area and has a user-friendly syntax and vast libraries. Data hiding in Python is performed using the `__` double underscore before done prefix. This makes the class members non-public and isolated from the other classes.

Example:

Python3

```
class Solution:
    __privateCounter = 0
    def sum(self):
        self.__privateCounter += 1
        print(self.__privateCounter)
count = Solution()
count.sum()
count.sum()
# Here it will show error because it unable
# to access private member
print(count.__privateCount)
```

Output:

Traceback (most recent call last):

File `"/home/db01b918da68a3747044d44675be8872.py"`, line 11, in `<module>`

```
print(count.__privateCount)
```

AttributeError: 'Solution' object has no attribute `'__privateCount'`

To rectify the error, we can access the private member through the class name :

Python3

```
class Solution:
    __privateCounter = 0

    def sum(self):
        self.__privateCounter += 1
        print(self.__privateCounter)

count = Solution()
count.sum()
count.sum()

# Here we have accessed the private data
# member through class name.
print(count._Solution__privateCounter)
```

Output:

```
1
2
2
```

Advantages of Data Hiding:

1. It helps to prevent damage or misuse of volatile data by hiding it from the public.
2. The class objects are disconnected from the irrelevant data.
3. It isolates objects as the basic concept of OOP.
4. It increases the security against hackers that are unable to access important data.

Disadvantages of Data Hiding:

1. It enables programmers to write lengthy code to hide important data from common clients.
2. The linkage between the visible and invisible data makes the objects work faster, but data hiding prevents this linkage.

Python Module and Import class

A Python module is a file containing Python definitions and statements. A module can define functions, classes, and variables. A module can also include runnable code. Grouping related code into a module makes the code easier to understand and use. It also makes the code logically organized.

Create a simple Python module

```
# A simple module, calc.py
```

```
def add(x, y):  
    return (x+y)  
  
def subtract(x, y):  
    return (x-y)
```

Import Module in Python

We can import the functions, and classes defined in a module to another module using the import statement in some other Python source file.

When the interpreter encounters an import statement, it imports the module if the module is present in the search path. A search path is a list of directories that the interpreter searches for importing a module. For example, to import the module `calc.py`, we need to put the following command at the top of the script.

Syntax of Python Import

```
import module
```

Note: This does not import the functions or classes directly instead imports the module only. To access the functions inside the module the dot(.) operator is used.

Importing modules in Python

Now, we are importing the **calc** that we created earlier to perform add operation.

```
# importing module calc.py  
import calc  
print(calc.add(10, 2))
```

Output:

12

The from-import Statement in Python

Python's *from* statement lets you import specific attributes from a module without importing the module as a whole.

Importing specific attributes from the module

Here, we are importing specific `sqrt` and `factorial` attributes from the `math` module.

```
# importing sqrt() and factorial from the
# module math
from math import sqrt, factorial
# if we simply do "import math", then
# math.sqrt(16) and math.factorial()
# are required.
print(sqrt(16))
print(factorial(6))
```

Output:

4.0

720

Import all Names

The * symbol used with the from import statement is used to import all the names from a module to a current namespace.

Syntax:

from module_name import *

From import * Statement

The use of * has its advantages and disadvantages. If you know exactly what you will be needing from the module, it is not recommended to use *, else do so.

```
# importing sqrt() and factorial from the
# module math
from math import *

# if we simply do "import math", then
# math.sqrt(16) and math.factorial()
# are required.
print(sqrt(16))
print(factorial(6))
```

Output

4.0

720

Locating Python Modules

Whenever a module is imported in Python the interpreter looks for several locations. First, it will check for the built-in module, if not found then it looks for a list of directories defined in the `sys.path`. Python interpreter searches for the module in the following manner –

- First, it searches for the module in the current directory.
- If the module isn't found in the current directory, Python then searches each directory in the shell variable `PYTHONPATH`. The `PYTHONPATH` is an environment variable, consisting of a list of directories.
- If that also fails python checks the installation-dependent list of directories configured at the time Python is installed.

Directories List for Modules

Here, `sys.path` is a built-in variable within the `sys` module. It contains a list of directories that the interpreter will search for the required module.

Python3

```
# importing sys module
import sys

# importing sys.path
print(sys.path)
```

Output:

```
['/home/nikhil/Desktop/gfg', '/usr/lib/python38.zip', '/usr/lib/python3.8',
'/usr/lib/python3.8/lib-dynload', '', '/home/nikhil/.local/lib/python3.8/site-packages',
'/usr/local/lib/python3.8/dist-packages', '/usr/lib/python3/dist-packages',
'/usr/local/lib/python3.8/dist-packages/IPython/extensions', '/home/nikhil/.ipython']
```

Renaming the Python module

We can rename the module while importing it using the keyword.

Syntax: *Import Module_name as Alias_name*

Python3

```
# importing sqrt() and factorial from the
# module math
import math as mt

# if we simply do "import math", then
# math.sqrt(16) and math.factorial()
# are required.
print(mt.sqrt(16))
print(mt.factorial(6))
```

Output

4.0